

Procesamiento Masivo de Datos

Linux, tuberías, expresiones regulares, `sed` y `awk`

Alex Di Genova

Universidad de O'Higgins | 2026

1 Motivación

La idea central del curso

Datos

GB → TB → PB

Herramientas

pequeñas y componibles

Objetivo

flujos reproducibles

Pregunta guía

¿Cómo procesamos archivos grandes **sin abrirlos completos en memoria**, evitando pasos manuales y dejando un análisis que pueda repetirse?

En esta clase, la terminal no es el objetivo: es el lenguaje común para mover, inspeccionar, transformar y resumir datos.

Objetivos de aprendizaje

Linux y shell

- navegar y manipular archivos
- comprender rutas, permisos y procesos
- usar redirecciones y tuberías

Procesamiento de texto

- buscar patrones con regex y `grep`
- transformar flujos con `sed`
- filtrar, calcular y resumir con `awk`

Meta de la clase

Convertir una pregunta sobre datos en una tubería verificable y reutilizable.

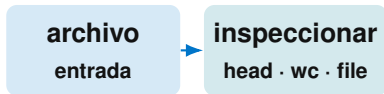
Del archivo al resultado: un modelo mental

archivo
entrada

Principio Unix

Resolver problemas complejos combinando herramientas simples, cada una especializada en una tarea pequeña.

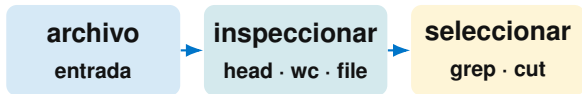
Del archivo al resultado: un modelo mental



Principio Unix

Resolver problemas complejos combinando herramientas simples, cada una especializada en una tarea pequeña.

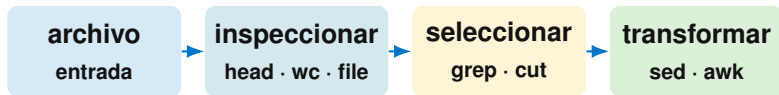
Del archivo al resultado: un modelo mental



Principio Unix

Resolver problemas complejos combinando herramientas simples, cada una especializada en una tarea pequeña.

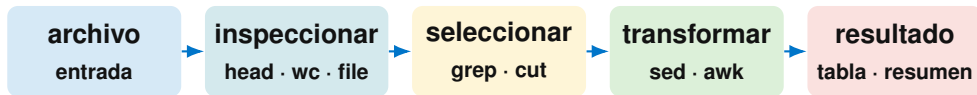
Del archivo al resultado: un modelo mental



Principio Unix

Resolver problemas complejos combinando herramientas simples, cada una especializada en una tarea pequeña.

Del archivo al resultado: un modelo mental



Principio Unix

Resolver problemas complejos combinando herramientas simples, cada una especializada en una tarea pequeña.

2 Linux y shell: lo esencial

¿Qué ocurre cuando escribimos un comando?

usuario: terminal · shell · programas

Idea clave

La **shell** interpreta el comando y crea procesos; el **kernel** administra los recursos físicos.

¿Qué ocurre cuando escribimos un comando?

usuario: terminal · shell · programas

llamadas al sistema

Idea clave

La **shell** interpreta el comando y crea procesos; el **kernel** administra los recursos físicos.

¿Qué ocurre cuando escribimos un comando?

usuario: terminal · shell · programas

llamadas al sistema

kernel: procesos · memoria · red · sistema de archivos

Idea clave

La **shell** interpreta el comando y crea procesos; el **kernel** administra los recursos físicos.

¿Qué ocurre cuando escribimos un comando?

usuario: terminal · shell · programas

llamadas al sistema

kernel: procesos · memoria · red · sistema de archivos

CPU · RAM · discos · interfaces de red

Idea clave

La **shell** interpreta el comando y crea procesos; el **kernel** administra los recursos físicos.

La shell es interfaz y también lenguaje

Como interfaz

- ejecuta programas
- conecta entrada y salida
- controla procesos

Como lenguaje

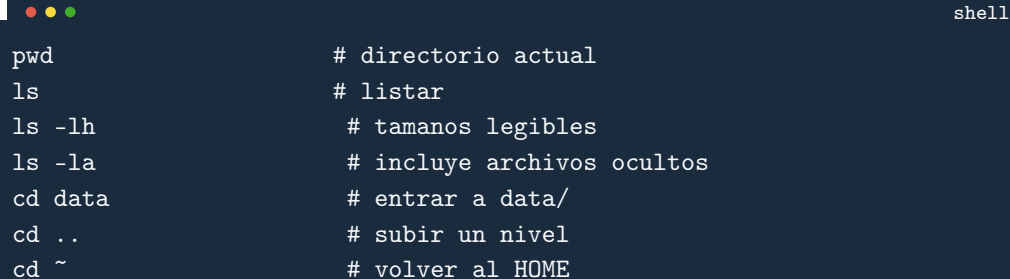
- variables y condiciones
- ciclos y funciones
- scripts versionables

```
$ pwd  
/home/alex/project  
$ command --option input.tsv
```

programa command **opción** -option

argumento input.tsv

Navegar: saber dónde estoy y qué hay alrededor



```
pwd          # directorio actual
ls           # listar
ls -lh       # tamanos legibles
ls -la       # incluye archivos ocultos
cd data      # entrar a data/
cd ..        # subir un nivel
cd ~         # volver al HOME
```

`.` = aquí

`..` = directorio padre

`~` = directorio personal

Archivos y directorios: crear, copiar, mover, borrar

crear y copiar

```
mkdir results  
mkdir -p results/qc  
cp input.tsv backup.tsv  
cp -r data/ data_backup/
```

mover y borrar

```
mv old.tsv new.tsv  
mv new.tsv results/  
rm file.tmp  
rm -r directory/
```

Atención `rm` no tiene papelera. Revise rutas y comodines antes de ejecutar.

Encontrar y medir datos

```
find data/ -type f -name '*.tsv'  
find . -type f -size +1G  
  
du -sh data/  
du -h --max-depth=1 data/ | sort -h  
  
df -h
```

Dos preguntas distintas

du pregunta: “¿cuánto ocupa este directorio?”
espacio queda en el sistema de archivos?”

df pregunta: “¿cuánto

Comodines y comillas: la shell expande antes de ejecutar

Globbering

```
ls *.tsv  
ls sample?.txt  
ls run_[123].log
```

Comillas

```
echo "$HOME"  
echo '$HOME'  
rm -- "file with spaces.txt"
```

Regla práctica

Use comillas dobles alrededor de variables y rutas cuando no quiera que espacios o comodines cambien el significado del comando.

Pedir ayuda es parte del trabajo

```
man sort  
sort --help  
command -v awk  
type cd
```

man

manual completo

-help

recordatorio
rápido

command -v

¿qué
ejecutable usaré?

Permisos: quién puede hacer qué

```
$ ls -l analysis.sh  
-rwxr-x--- 1 alex lab 2048 Aug 31 analysis.sh
```

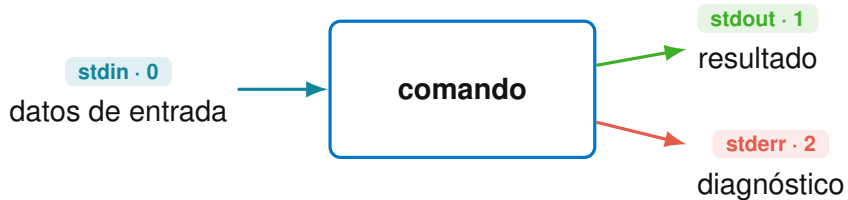
	usuario	grupo	otros
representación	rwX	r-X	—
significado	leer/escribir/ejecutar	leer/ejecutar	ninguno

mínimo privilegio Conceda sólo el acceso necesario; evite `chmod 777` como solución rápida.

3

Flujos, redirecciones y tuberías

Tres flujos; infinitas combinaciones



```
command < input.txt > result.txt 2> errors.log
```

Redirecciones: decidir de dónde vienen y adónde van los datos

```
command > result.txt           # sobrescribe stdout
command >> result.txt          # agrega stdout al final
command 2> errors.log          # guarda stderr
command > out.txt 2>&1         # stdout + stderr
command < input.txt            # toma stdin desde archivo
```

Consejo Antes de usar `>`, confirme que no está sobrescribiendo un resultado importante.

La tubería: stdout de un proceso → stdin del siguiente

pipeline.sh

```
cat file.txt \  
| grep 'ERROR' \  
| sort \  
| uniq -c \  
| sort -nr
```

Regla de depuración

Construya la tubería de izquierda a derecha. Ejecute cada etapa sobre una muestra pequeña antes de agregar la siguiente.

No siempre necesitamos `cat`

Funciona

```
cat file.tsv | grep chile
```

Más directo

```
grep chile file.tsv
```

Pero `cat` es útil para concatenar archivos, alimentar comandos que sólo leen stdin o hacer una tubería más explícita durante la enseñanza.

4 Inspeccionar, seleccionar y ordenar

Inspeccionar antes de transformar

```
head -n 5 worldcitiespop.csv  
tail -n 5 worldcitiespop.csv  
wc -l worldcitiespop.csv  
file worldcitiespop.csv  
less worldcitiespop.csv
```

1 ¿hay cabecera?

2 ¿cuál es el
delimitador?

3 ¿qué valores
faltan?

Nunca empiece transformando un archivo que todavía no ha inspeccionado.

Una caja de herramientas para texto

inspeccionar	<code>head, tail, less, wc, file</code>
buscar	<code>grep</code>
columnas	<code>cut, paste</code>
ordenar/agrupar	<code>sort, uniq</code>
transformar	<code>tr, sed, awk</code>
combinar/comparar	<code>join, comm, diff</code>
encontrar archivos	<code>find</code>

Elija la herramienta más simple que exprese correctamente la operación.

Seleccionar columnas con `cut`

worldcitiespop.csv

```
head -n 3 worldcitiespop.csv
```

```
cut -d, -f1,2,5 worldcitiespop.csv | head
```

Importante

`cut -d,` sirve aquí porque el archivo es simple. Un CSV general puede contener comas entre comillas y necesita un parser real.

Ordenar y contar categorías

```
cut -d, -f1 worldcitiespop.csv \  
| tail -n +2 \  
| sort \  
| uniq -c \  
| sort -nr \  
| head
```

sort agrupa iguales + **uniq -c** cuenta adyacentes + **sort -nr** ordena por frecuencia

Caso guiado: las 10 ciudades más pobladas

Pregunta: ¿cuáles son las 10 ciudades con mayor población?

top-cities.sh

```
tail -n +2 worldcitiespop.csv \  
  | sort -t, -k5,5nr \  
  | head -n 10
```

quitar cabecera



ordenar campo 5 numérico



tomar diez

Procesar datos comprimidos sin descomprimirlos a disco

```
gzip -dc reads.txt.gz | head
zcat table.tsv.gz | wc -l

zcat table.tsv.gz \
| grep -v '^#' \
| cut -f1,2 \
| head
```

Por qué importa

En datos masivos, evitar copias intermedias ahorra espacio, I/O y tiempo.

Muchas herramientas pueden leer directamente desde [stdin](#).

¿Qué operaciones escalan bien?

Streaming

`grep`, `cut`, `sed` y muchos usos de `awk` procesan una línea a la vez.

Requieren más recursos

`sort` necesita ordenar globalmente; agregaciones en `awk` pueden almacenar muchas claves en memoria.

Pregunta de escalabilidad

¿cuánto crece la memoria y el disco temporal cuando crece el archivo?

5

Expresiones regulares y grep

Regex: describir patrones, no posiciones fijas

Patrón	Significado
<code>^</code> <code>\$</code>	inicio y final de línea
<code>.</code>	cualquier carácter
<code>[abc]</code>	uno de los caracteres del conjunto
<code>[^abc]</code>	cualquier carácter excepto los del conjunto
<code>*</code> <code>+</code> <code>?</code>	cero o más; uno o más; opcional
<code>{m,n}</code>	entre <i>m</i> y <i>n</i> repeticiones
<code>(a b)</code>	agrupación y alternativas en ERE
<code>[[:digit:]]</code>	clase portable de dígitos

Consejo

Use `grep -E` y `sed -E` para expresiones regulares extendidas.

Buscar con grep

```
grep 'Chile' countries.txt  
grep -i 'chile' countries.txt  
grep -n 'ERROR' pipeline.log  
grep -v '^#' variants.tsv  
grep -E '^(chr1|chr2)[[:space:]]' variants.tsv
```

i ignora caso

n muestra línea

v invierte

E regex
extendida

Regex en acción

```
regex-demo.sh
$ printf '%s\n' ac abc abbc abbbc abbbbbc \
  | grep -E '^ab{1,3}c$'
abc
abbc
abbbc
```

Leer el patrón de izquierda a derecha

inicio · a · entre 1 y 3 letras b · c · final de línea

6 sed: transformar un flujo

sed: pensar en “buscar y transformar”

Sustituir



sed · sustitución

```
sed 's/hello/bye/' file.txt  
sed 's/hello/bye/g' file.txt
```

Normalizar espacios



sed · regex extendida

```
sed -E 's/[[[:space:]]+]/ /g' file.  
txt
```

s/patrón/reemplazo/banderas

sed: **seleccionar líneas y rangos**

```
sed -n '10,20p' file.txt  
sed -n '/ERROR/p' pipeline.log  
sed '10,20s/hello/bye/g' file.txt  
sed '/^#/d' variants.tsv
```

p

imprimir

d

eliminar

10,20

rango de líneas

Capturas: reorganizar texto con grupos

```
sed · grupos de captura  
$ echo 'chr1:100-250' \  
  | sed -E 's/([^:]+):([0-9]+)-([0-9]+)/\1\t\2\t\3/'  
chr1      100      250
```

Idea clave

Los paréntesis capturan partes del patrón; \1, \2, \3 las reutilizan en el reemplazo.

Ejemplos de transformación con sed

```
sed 's/2022/2024/g' file.txt
sed 's/2022/2024/g; s/casa/home/g' file.txt

echo 'cats dog bee parrot foxed' \
  | sed -E 's/cat|dog|fox/--/g'
# --s -- bee parrot --ed

echo 'car bat cod map scat dot abacus' \
  | sed 's/b.*at/-/'
# car - dot abacus
```

Discusión: `.*` es codicioso; consume todo lo posible antes de permitir que el resto del patrón coincida.

7 awk: filtrar, calcular y resumir

awk: datos como registros y campos

- un registro por línea
- campos `$1 ... $NF`
- `NR`: número de registro
- regla = patrón + acción

```
awk -F, 'NR > 1 && $5 > 1e6 {  
    print $2, $5  
}' worldcitiespop.csv
```

`awk 'condición { acción }' archivo`

Separadores: FS para leer, OFS para escribir

```
awk -F, 'BEGIN {OFS="\t"}  
  NR > 1 {print $1, $2, $5}  
' worldcitiespop.csv | head
```

Modelo

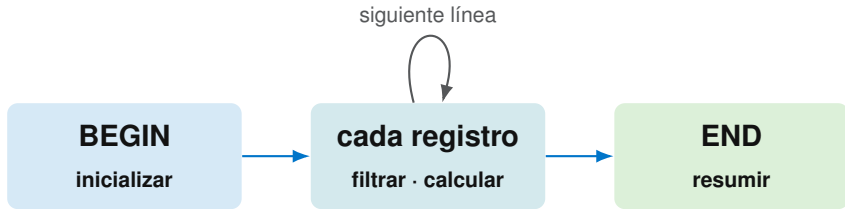
FS define cómo se separan los campos de entrada; OFS define cómo se separan los campos impresos.

Patrones y condiciones con `awk`

```
awk '$NF < 0' table.txt  
awk 'NR == 1 || $5 > 1000000' worldcitiespop.csv  
awk '$2 ~ /error/' log.txt  
awk 'tolower($2) ~ /error/' log.txt  
awk 'NF >= 5 {print $1, $NF}' table.txt
```

`~` coincide con regex `!~` no coincide `NF` número de campos

El ciclo de vida de `awk`



```
awk 'BEGIN {sum=0}
     {sum += $1}
     END {print sum}' values.txt
```


Agregación con arreglos asociativos

Pregunta: ¿cuál es la población total registrada por país?

population-by-country.sh

```
awk -F, '
  NR > 1 && $5 != "" { pop[$1] += $5 }
  END { for (c in pop) print c, pop[c] }
' worldcitiespop.csv \
| sort -k2,2nr \
| head
```

clave país + **valor** suma acumulada = **resumen** por grupo

Contar categorías con `awk`

```
awk -F, '
  NR > 1 { count[$1]++ }
  END {
    for (c in count)
      print c, count[c]
  }
' worldcitiespop.csv \
| sort -k2,2nr \
| head
```

El patrón `count[key]++` aparece constantemente en logs, tablas, variantes, muestras y resultados de pipelines.

Funciones útiles de `awk`

Cadenas

- `length(s)`, `substr(s,p,n)`
- `split(s,a,sep)`
- `sub`, `gsub`, `match`
- `tolower`, `toupper`

Cálculo

- `+` `-` `*` `/` `%` `^`
- `sqrt`, `log`, `int`
- `==` `!=` `<` `<=` `>` `>=`
- `&&` `||` `!`

Laboratorio corto: selección con `awk`

```
$ cat table.txt
brown bread mat hair 42
blue cake mug shirt -7
yellow banana window shoes 3.14

$ awk '{print $2}' table.txt
bread
cake
banana

$ awk '$NF < 0' table.txt
blue cake mug shirt -7

$ awk '{gsub(/b/, "B", $1)} 1' table.txt
Brown bread mat hair 42
Blue cake mug shirt -7
yellow banana window shoes 3.14
```

8 Combinar herramientas

Ejemplo integrado: frecuencia de palabras

word-frequency.sh

```
tr '[:upper:]' '[:lower:]' < text.txt \  
| tr -cs '[:alnum:]' '\n' \  
| sed '/^$/d' \  
| sort \  
| uniq -c \  
| sort -nr \  
| head -n 20
```

Discuta: ¿qué ocurre con tildes, guiones, números y distintas configuraciones regionales?

Ejemplo integrado: resumir un archivo de variantes

Supongamos un TSV simplificado: CHROM POS REF ALT SAMPLE

```
variants.tsv
grep -v '^#' variants.tsv \
  | awk 'BEGIN {OFS="\t"}
        {count[$1]++}
        END {for (chr in count) print chr, count[chr]}' \
  | sort -k2,2nr
```

Transferencia de aprendizaje

El mismo patrón sirve para contar eventos por cromosoma, errores por nodo, muestras por grupo o registros por categoría.

Combinar dos tablas con `join`

`join` requiere que ambas tablas estén ordenadas por la clave.

```
sort -k1,1 countries.tsv > countries.sorted.tsv  
sort -k1,1 population.tsv > population.sorted.tsv  
  
join -t $'\t' -1 1 -2 1 \  
    countries.sorted.tsv population.sorted.tsv
```

Concepto

Es el equivalente en shell de un *inner join* simple.

Una tubería reproducible deja huellas

- script versionado

```
#!/usr/bin/env bash
set -euo pipefail

pipeline input.tsv > result.tsv 2> pipeline.log
```

Una tubería reproducible deja huellas

- script versionado
- entradas identificadas

```
#!/usr/bin/env bash
set -euo pipefail

pipeline input.tsv > result.tsv 2> pipeline.log
```

Una tubería reproducible deja huellas

- script versionado
- entradas identificadas
- parámetros explícitos

```
#!/usr/bin/env bash  
set -euo pipefail
```

```
pipeline input.tsv > result.tsv 2> pipeline.log
```

Una tubería reproducible deja huellas

- script versionado
- entradas identificadas
- parámetros explícitos
- versiones de herramientas

```
#!/usr/bin/env bash
set -euo pipefail

pipeline input.tsv > result.tsv 2> pipeline.log
```

Una tubería reproducible deja huellas

- script versionado
- entradas identificadas
- parámetros explícitos
- versiones de herramientas
- errores registrados

```
#!/usr/bin/env bash
set -euo pipefail

pipeline input.tsv > result.tsv 2> pipeline.log
```

Una tubería reproducible deja huellas

- script versionado
- entradas identificadas
- parámetros explícitos
- versiones de herramientas
- errores registrados
- conteos de control

```
#!/usr/bin/env bash
set -euo pipefail

pipeline input.tsv > result.tsv 2> pipeline.log
```

Una tubería reproducible deja huellas

- script versionado
- entradas identificadas
- parámetros explícitos
- versiones de herramientas
- errores registrados
- conteos de control
- salidas deterministas

```
#!/usr/bin/env bash
set -euo pipefail

pipeline input.tsv > result.tsv 2> pipeline.log
```

Una tubería reproducible deja huellas

- script versionado
- entradas identificadas
- parámetros explícitos
- versiones de herramientas
- errores registrados
- conteos de control
- salidas deterministas
- comando de ejecución documentado

```
#!/usr/bin/env bash
set -euo pipefail

pipeline input.tsv > result.tsv 2> pipeline.log
```


Ver el flujo mientras se ejecuta: tee

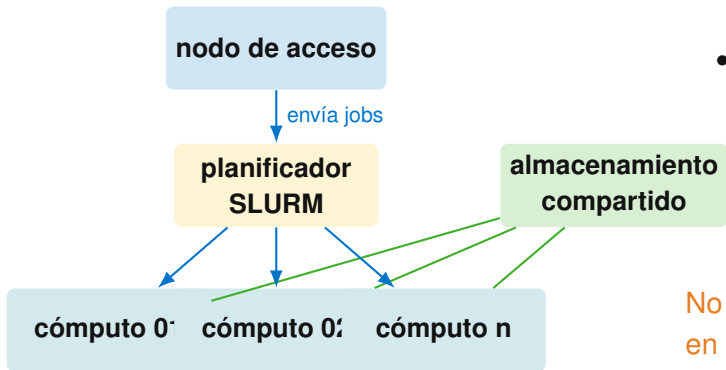
```
grep -v '^#' variants.tsv \  
  | tee filtered.tsv \  
  | awk '{count[$1]++}  
        END {for (c in count) print c, count[c]}'
```

Útil para depurar

tee copia el flujo a un archivo y, al mismo tiempo, lo deja continuar por la tubería.

9 Trabajo remoto y clúster

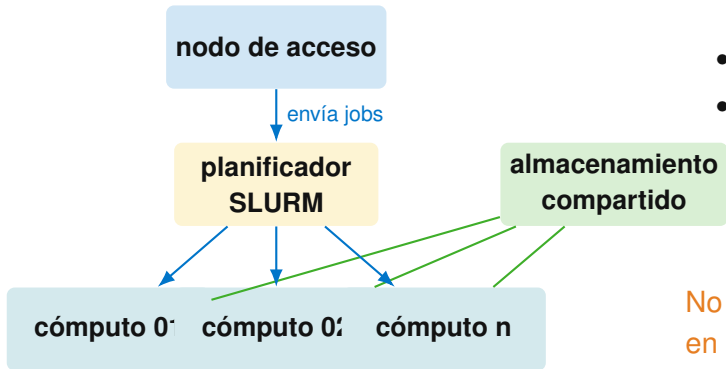
El clúster: dónde ejecutaremos los análisis grandes



- SSH para entrar

No ejecute análisis pesado en el nodo de acceso.

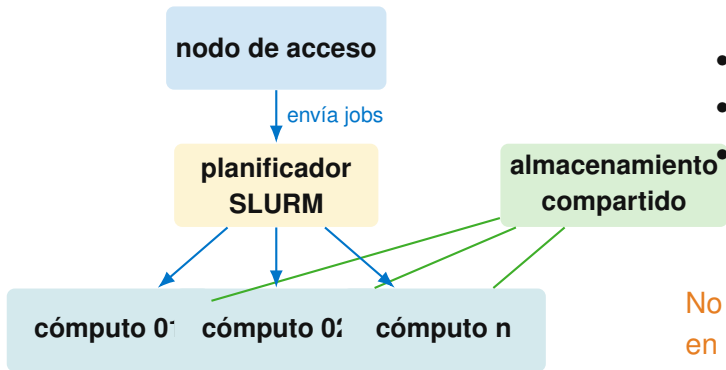
El clúster: dónde ejecutaremos los análisis grandes



- SSH para entrar
- SLURM para computar

No ejecute análisis pesado en el nodo de acceso.

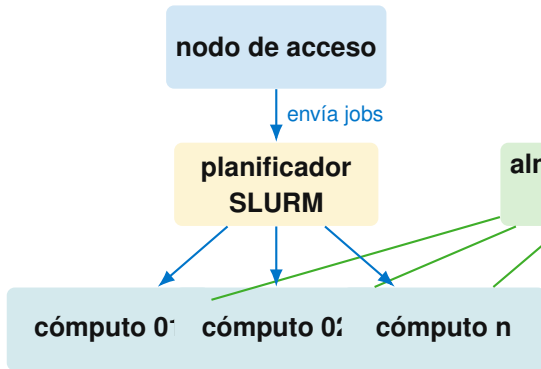
El clúster: dónde ejecutaremos los análisis grandes



- SSH para entrar
- SLURM para computar
- filesystem compartido

No ejecute análisis pesado en el nodo de acceso.

El clúster: dónde ejecutaremos los análisis grandes



- SSH para entrar
- SLURM para computar
- filesystem compartido
- nodos para trabajo pesado

No ejecute análisis pesado en el nodo de acceso.

Acceso remoto y transferencia

Conectar

```
ssh user@cluster  
ssh -p 8765 user@cluster
```

Transferir

```
scp file.tsv user@cluster:/data/  
rsync -avP results/ \  
user@cluster:/data/results/
```

rsync especialmente útil para directorios grandes y transferencias que pueden reanudarse.

Sesiones persistentes para trabajo interactivo

```
tmux new -s analysis

# Ctrl-b d    -> separar la sesion

tmux ls
tmux attach -t analysis
```

En un clúster

Use `tmux` para sesiones interactivas livianas; use el planificador para análisis que consumen CPU o memoria de manera significativa.

10 Práctica

Laboratorio: ciudades del mundo

worldcitiespop.csv

```
head worldcitiespop.csv
wc -l worldcitiespop.csv
head -100 worldcitiespop.csv > sample_worldcitiespop.csv

cut -d, -f1,5 worldcitiespop.csv > country_population.csv

tail -n +2 worldcitiespop.csv \
  | sort -t, -k5,5nr \
  | head -10

awk -F, 'NR > 1 && $5 > 1000000' worldcitiespop.csv
```

Laboratorio: de pregunta a solución

1. ¿Cuántas filas tiene el archivo y cuántas poseen población informada?
2. ¿Cuántos países distintos aparecen?
3. ¿Cuáles son las 10 ciudades más pobladas?
4. ¿Cuánta población total registra cada país?
5. ¿Qué países tienen más ciudades con población > 1 millón?

Criterio de evaluación

Cada respuesta debe incluir: comando, resultado, una comprobación y una explicación de una línea.

Desafíos: de simple a compuesto

1. Contar la frecuencia de palabras en un texto.
2. Reemplazar `hello` por `bye` sólo entre las líneas 10 y 20.
3. Extraer un rango `chr:start-end` y convertirlo a tres columnas.
4. Combinar dos tablas por un campo común.
5. Contar registros por categoría con un arreglo asociativo de `awk`.
6. Procesar un archivo `.gz` sin crear una copia descomprimida.

Criterio

Primero una prueba pequeña; después el archivo completo.

11

Herramientas y frontera práctica

¿Cuándo dejar shell, sed o awk?

Shell

orquestrar programas,
archivos y procesos.

sed / awk

transformar y resumir texto
estructurado de forma
compacta.

Python / R

lógica extensa, parsers
complejos, modelos y
visualización.

Frontera práctica

Una tubería debe ser fácil de leer. Si la lógica deja de ser obvia, muévela a un script más expresivo.

CSV y tablas modernas

csvtk

filtrar, mutar y combinar CSV/TSV dentro de tuberías.

xsv

seleccionar, indexar, unir y analizar CSV rápidamente.

YouPlot

inspeccionar distribuciones directamente en terminal.

Shell sigue siendo la capa de composición; las herramientas especializadas resuelven formatos que requieren un parser real.

Recursos del curso

- Linux Upskill Challenge: <https://linuxupskillchallenge.org/>
- GNU awk: <https://www.gnu.org/software/gawk/manual/>
- GNU sed: <https://www.gnu.org/software/sed/manual/>
- Learn GNU awk: https://learnbyexample.github.io/learn_gnuawk/
- Learn GNU sed: https://learnbyexample.github.io/learn_gnused/
- Regex101: <https://regex101.com/>
- worldcitiespop.csv: <https://burntsushi.net/stuff/worldcitiespop.csv>
- csvtk: <https://github.com/shenwei356/csvtk>

Ideas para llevar

1. Inspeccione antes de transformar.
2. Piense en datos como flujos: `stdin`, `stdout`, `stderr`.
3. Combine herramientas pequeñas con tuberías.
4. `grep` busca; `sed` transforma; `awk` filtra, calcula y resume.
5. Pregunte siempre cómo escalan memoria, disco e I/O.
6. Convierta el análisis final en un script reproducible.

Siguiente paso

Tome una tarea manual, exprésela como tubería, pruébela con una muestra pequeña y luego ejecútela sobre el conjunto completo.

Preguntas